DSA03-NATURAL LANGUAGE PROCESSING
ASSIGNMENT-2

# ASSIGNMENT 2 – CO2

**CO2 Coverage:** N-grams Models of Syntax; Counting Words; Unsmoothed N-grams; Smoothing — Backoff, Deleted Interpolation; Entropy; English Word Classes; Tagsets for English; Part of Speech Tagging; Rule-Based Part of Speech Tagging; Stochastic Part of Speech Tagging; Transformation-Based Tagging

**Nature of Assignment:** Individual
**Duration:** 1 Week
**Total Marks:** 50
**Submission Date:** 05/11/26

**Assignment Title: Build and Evaluate an N-gram Language Model and a Part-of-Speech Tagger for a Public Transport Enquiry Helpline (Total Marks – 50)**

Duration: 1 week

**Scenario — Speech Transcript Analytics for a Transport Enquiry Helpline**

A state transport corporation runs a telephone enquiry helpline that takes about four thousand calls a day about bus and train timings, routes, fares, cancellations and refunds. The calls are transcribed automatically, and the speech recogniser does not return one transcript but several competing hypotheses for every utterance: "two tickets to Vellore", "to tickets to Vellore" and "two tickets to Velore" all come back with similar acoustic scores. The recogniser cannot choose between them, because the decision is linguistic, not acoustic. Once a transcript is chosen it must be tagged before the origin, destination, date and fare can be pulled out of it, since "book", "return" and "stand" are each a noun in one call and a verb in the next.

You are the language-engineering team. You must build the n-gram language model that rescores the recogniser's hypotheses, measure it honestly with entropy and perplexity on held out data, design a tagset for the domain, and build and compare three part-of-speech taggers — rule-based, stochastic and transformation-based. Every claim you make about one model being better than another must rest on a measurement you made yourself.

**Planning data and constraints**

- Corpus: at least 20,000 transcribed utterances totalling at least 150,000 word tokens (real or

synthetic, but documented), split 80 / 10 / 10 into training, development and test sets with no utterance appearing in more than one split.

• Recogniser output: at least 200 utterances for which five competing hypotheses and the correct reference transcript are available, so that rescoring can be evaluated.

• Tagset (design and document your own): between 20 and 30 tags covering the open and closed word classes of the domain, with an explicit mapping to the Penn Treebank tagset.

• Annotated data: at least 3,000 tokens hand-tagged with your tagset, of which at least 500 tokens are tagged independently by two people so that agreement can be reported.

• Evaluation: a held-out set that is never used for training or for tuning any weight, and a fixed random seed for every experiment so that results can be reproduced.

• Any manual effort (transcription, tagset design, annotation, rule writing, testing) must be accounted for in your effort and cost discussion.

**Design Challenge (Day 4):**

The helpline is extended to metro and intercity rail services and begins to receive code-mixed speech, so the vocabulary grows sharply and more than fifteen per cent of the words in new calls are never seen in training. Re-evaluate your language model and your taggers and decide whether to retain, modify or redesign them.

**TASK 1 – Counting Words and Building the Vocabulary 3 Marks**

**Guiding Enquiry**

What exactly is being counted when we count the words of a corpus, and how do those decisions change every model built on top of them?

**Investigate and interpret:**

• Distinguish type, token, lemma and word form, and report all four counts for your corpus; state how you treated case, punctuation, numerals, disfluencies and the utterance boundary markers <s> and </s>.

• Justify every tokenisation decision that changes the counts — whether "Vellore" and "vellore" are one type, whether "6:30" is one token or three — and show the effect of two different decisions on the vocabulary size.

• Plot the frequency of each word against its rank, state Zipf's law, and check how closely your corpus follows it; identify the words that deviate most and explain why the domain causes the deviation.

• Build the vocabulary with an unknown-word threshold, mapping rare words to <UNK>; report the vocabulary size at thresholds of 1, 2, 3 and 5 and the proportion of tokens each threshold sends to <UNK>.

• Measure the out-of-vocabulary rate of the development set against the training vocabulary and explain why this single number limits everything that follows.

**Evidence Expected**

Documented counting decisions, a Zipf plot for the corpus, and vocabulary and OOV figures at several thresholds.

**TASK 2 – Unsmoothed N-gram Models 4 Marks**

**Guiding Enquiry**

How is the probability of a word sequence estimated from counts alone, and where does that estimate break down?

**Investigate and interpret:**

- State the chain rule for the probability of a word sequence and derive the Markov assumption that turns it into a bigram and then a trigram model, saying precisely what is being assumed away at each step.
- Give the maximum likelihood estimator for unigram, bigram and trigram probabilities, and build all three models from your training split.
- Compute by hand the unsmoothed probability of at least three helpline utterances, showing every factor; choose one utterance that is perfectly reasonable English but receives probability zero, and identify the exact bigram responsible.
- Report the sparsity of your models: the proportion of possible bigrams and trigrams actually observed, and the proportion of development-set bigrams and trigrams never seen in training.
- Use the models as generators: sample ten utterances from the unigram, bigram and trigram models and comment on what the increase in order buys and what it costs.

**Evidence Expected**

Working unsmoothed models, hand-computed sentence probabilities including a zero, and quantified sparsity figures.

**TASK 3 – Smoothing and Discounting 5 Marks**

**Guiding Enquiry**

How can probability mass be moved from what was seen to what was not, without destroying what the counts tell us?

**Investigate and interpret:**

- Explain why a zero probability is a modelling failure rather than a fact about the language, and state the condition any smoothing method must satisfy for the distribution to remain valid.
- Implement add-one (Laplace) and add-k smoothing; for a bigram with a left context of your choice, tabulate the original counts, the effective (discounted) counts and the

resulting probabilities, and show exactly how much mass was taken from the seen bigrams.

- Explain why add-one smoothing discounts so severely when the vocabulary is large, with the arithmetic for one context from your own corpus.
- Implement Good–Turing discounting: build the frequency-of-frequencies table $N_r$, compute the revised counts c*, estimate the probability of all unseen bigrams, and explain how the high-frequency tail is handled.
- Compare the methods on the development set by perplexity and by how reasonable the estimate for a specific unseen bigram looks, and state which you would deploy and why.

**Evidence Expected**

Correctly implemented smoothing methods with discount tables, a Good–Turing $N_r$ table, and a measured comparison.

**TASK 4 – Backoff and Deleted Interpolation 4 Marks**

**Guiding Enquiry**

When the trigram has never been seen, how should the model fall back on the shorter contexts it does know?

**Investigate and interpret:**

- Define backoff and interpolation and state the essential difference: backoff consults a shorter context only when the longer one fails, interpolation always combines all orders.
- Implement Katz backoff with Good–Turing discounting, explaining the role of the normalising weight $\alpha$ and why the model is not a valid distribution without it; also implement stupid backoff and say why its output is a score and not a probability.
- Implement simple linear interpolation, then deleted interpolation: hold out part of the training data, estimate the $\lambda$ weights on it, and report the weights you obtain for at least three context-count bands.
- Explain why the $\lambda$ weights should depend on the count of the context, and show with one worked example from your corpus how a low-count context shifts weight onto the lower orders.
- Compare backoff and interpolation on the held-out set by perplexity, and hand-trace one utterance through both so that the difference in treatment of a single unseen trigram is visible.

**Evidence Expected**

Working backoff and interpolation models, estimated $\lambda$ weights with the held-out procedure described, and a traced comparison.

**TASK 5 – Entropy, Cross-Entropy and Perplexity 4 Marks**

**Guiding Enquiry**

How is a language model measured, and what does the number actually mean?

**Investigate and interpret:**

- Define entropy, cross-entropy and perplexity, state the relation between them, and explain in plain words what a perplexity of 90 tells an engineer about the model.
- Compute the entropy of at least three distributions from your own model by hand — including one that is nearly uniform and one that is sharply peaked — and explain what the difference means for prediction.
- Compute the cross-entropy and perplexity of the held-out set under each model built in Tasks 2 to 4, and present the results in one comparable table; state the per-word and per-utterance figures separately.
- Explain why perplexity may only be compared between models that share a vocabulary and the same treatment of <UNK>, and demonstrate the distortion with a deliberately unfair comparison of your own.
- Use your 200 recogniser hypothesis sets: rescore each set with your best model, report the word error rate before and after rescoring, and state whether perplexity improvements translated into error reductions.

**Evidence Expected**

Correct entropy calculations, a comparable perplexity table, and a rescoring experiment relating perplexity to word error rate.

**TASK 6 – English Word Classes and Tagset Design 4 Marks**

**Guiding Enquiry**

What are the word classes of English, and how many distinctions should a tagset for this helpline actually make?

**Investigate and interpret:**

- Distinguish open and closed word classes, list the members of the closed classes that occur in your corpus, and explain why the closed classes matter disproportionately for tagging.
- Define the major open classes with the morphological and distributional tests used to identify them, and apply those tests to at least five ambiguous words from your corpus.
- Design your tagset of 20 to 30 tags, justify each distinction you chose to make and at least three you chose not to make, and give the full mapping to the Penn Treebank tagset.
- Annotate your 3,000-token sample, report the tag frequency distribution, and give the inter-annotator agreement on the doubly annotated portion as a raw percentage and as Cohen's kappa.
- Measure the ambiguity your tagset leaves: the proportion of word types and of word tokens

with more than one possible tag, and the average number of tags per ambiguous token.

**Evidence Expected**

A justified tagset with a Penn Treebank mapping, an annotated sample with agreement figures, and a measurement of residual ambiguity.

**TASK 7 – Rule-Based Part-of-Speech Tagging 4 Marks**

**Guiding Enquiry**

How far can hand-written rules take a tagger, and what makes writing them hard?

**Investigate and interpret:**

- Describe the two-stage architecture of a rule-based tagger: a dictionary that assigns every possible tag, then constraints that remove the impossible ones.
- Build the tag dictionary from your annotated data and report how many words receive one, two, three or more candidate tags.
- Write at least fifteen disambiguation rules in an explicit condition-action notation, each stating the context it inspects; include rules that use the preceding tag, the following word and the word's own suffix.
- Evaluate the tagger on the held-out set, report overall accuracy and accuracy on ambiguous tokens only, and explain why the second number is the one that matters.
- Analyse the failures: give the five commonest error types, show one case where two of your rules conflict, and explain how rule ordering resolved it and what that costs in maintainability.

**Evidence Expected**

A documented rule set with a tag dictionary, measured accuracy on ambiguous tokens, and a rule-conflict analysis.

**TASK 8 – Stochastic (HMM) Tagging and the Viterbi Algorithm 5 Marks**

**Guiding Enquiry**

How does a probabilistic tagger choose a whole tag sequence rather than one tag at a time?

**Investigate and interpret:**

- Derive the HMM tagging equation from Bayes' rule, state the two independence assumptions it rests on, and identify the transition and emission terms.
- Estimate the transition and emission probabilities from your annotated data, state how you smoothed them, and show that each transition row sums to one.
- Hand-trace the Viterbi algorithm on one ambiguous helpline utterance of at least five words: fill every trellis cell, keep a back-pointer for each, recover the best tag sequence

and give its probability.

- Compare Viterbi against the most-frequent-tag baseline and against scoring every tag sequence exhaustively; report the accuracy of each and the number of operations the three require for your sentence.
- Explain how unknown words are tagged by an HMM — suffix evidence, capitalisation, an open-class prior — implement one such method, and report its accuracy on the out of-vocabulary tokens alone.

**Evidence Expected**

A correctly estimated HMM, a fully traced Viterbi trellis with back-pointers, and a measured treatment of unknown words.

**TASK 9 – Transformation-Based (Brill) Tagging 4 Marks**

**Guiding Enquiry**

Can a tagger learn its own correction rules, and are the rules it learns worth reading?

**Investigate and interpret:**

- Describe transformation-based learning: the initial-state annotator, the templates, the greedy selection of the transformation that corrects the most errors, and the stopping condition.
- Define at least six rule templates over the preceding tag, the following tag, the surrounding words and the word's own form, and justify the window you chose.
- Run the learner from a most-frequent-tag initial state and report the first ten transformations learnt in order, with the number of errors each one corrected and introduced.
- Explain why the learnt rules are readable in a way that HMM probabilities are not, and show one learnt rule that encodes a genuine linguistic generalisation and one that is an artefact of your training data.
- Measure accuracy against the number of transformations applied, plot the curve, and identify the point beyond which further rules overfit.

**Evidence Expected**

A working Brill learner with its learnt rule list, and an accuracy curve identifying the overfitting point.

**TASK 10 – Implement and Test the Models 3 Marks**

**Guiding Enquiry**

Can a working implementation reproduce the behaviour you derived by hand?

**Investigate and interpret:**

- Implement in Python the counting and vocabulary pipeline, the unigram, bigram and trigram models with every smoothing method, backoff and interpolation, and all three taggers, using the standard library and NLTK only.
- Run every model over the same splits and report, for each: training time, model size in memory, held-out perplexity or tagging accuracy, and tagging speed in tokens per second.
- Include test cases for the zero-probability utterance from Task 2, the λ weights from Task 4, the trellis from Task 8 and the first transformations from Task 9, checking each against your hand-computed result.
- Present the results in comparable tables and at least two plots — for example perplexity against n-gram order, and tagging accuracy against training-set size.

**Evidence Expected**

Working, documented code, a reproducible test procedure, and results tables and graphs that match the hand calculations.

**TASK 11 – Compare, Recommend and Defend the Design 2 Marks**

**Guiding Enquiry**

Which language model and which tagger would you deploy on the helpline, and what evidence supports the choice?

**Investigate and interpret:**

- Compare the smoothing and backoff strategies on perplexity, word error rate after rescoring, memory and speed, and compare the three taggers on accuracy, accuracy on ambiguous and unknown tokens, speed and ease of maintenance.
- Recommend one language model and one tagger, state the expected accuracy, the failure modes, and what a caller would actually experience when the system fails.
- Discuss the manual effort involved in annotation and rule writing, and identify the point at which more annotated data would help more than more rules.

**Evidence Expected**

A justified recommendation supported by theory and measurements; choosing the highest accuracy alone is not sufficient.

**DAY-4 DESIGN CHALLENGE**

**TASK 12 – Re-evaluate and Adapt the Design 8 Marks**

**Changed Requirement**

The helpline now also answers metro and intercity rail enquiries, so hundreds of new station names, train names and fare categories enter the vocabulary every month. Callers increasingly speak code-mixed sentences, for example "Chennai to Madurai bus timing enna", and the recogniser transcribes those words phonetically and inconsistently. More than fifteen per cent of the word tokens in new calls were never seen in training, many of them exactly the proper nouns that carry the caller's intent.

Re-evaluate your original design under the changed conditions. You may:

**RETAIN → MODIFY → or REDESIGN**

your original solution.

**A. Impact Analysis – 2 Marks**

**Guiding Enquiry**

What exactly has changed, and which of your original assumptions no longer hold?

- Identify what the new input breaks: a closed vocabulary, a stable unknown-word rate, English-only word classes, transition probabilities estimated on a narrower domain, and a tagset that has no category for code-mixed tokens.
- Explain why the out-of-vocabulary words are the worst possible ones to lose, given that they are mostly the place names the enquiry turns on.
- Estimate the effect on perplexity and on tagging accuracy, reasoning from the OOV rate you measured in Task 1 and the unknown-word accuracy you measured in Task 8.

**Evidence Expected**

A clear analysis of how the new input exposes gaps in the original design.

**B. Evaluation of the Original Design – 2 Marks**

**Guiding Enquiry**

How would your recommended model and tagger behave on the new calls?

- Run or hand-trace at least three new utterances — one with an unseen station name, one code-mixed, one with both — through your language model and your chosen tagger, and record what fails first and why.
- Measure the degradation: held-out perplexity, word error rate after rescoring, and tagging accuracy on the new sample against your original baselines.
- Identify which component degrades fastest, and whether a different approach — a class-based or character-level model, an open-vocabulary tagger, a gazetteer of station names — would handle the new input more economically.

**Evidence Expected**

A technically supported evaluation of the original design with traces or measurements.

**C. Adaptation or Redesign – 2 Marks**

**Guiding Enquiry**

Should the models be retrained, extended or replaced?

- Choose retain, modify or redesign — for example adding a station-name gazetteer and a class-based n-gram over it, re-estimating λ weights on new data, extending the tagset with a code-mixed category, or adding a character n-gram model for unseen words.
- Describe the revised design and state exactly what changes in the vocabulary, the smoothing, the backoff weights, the tagset and the tagger.
- Determine the effect on perplexity, tagging accuracy, retraining frequency and annotation effort, and say how often the model must now be refreshed.

**Evidence Expected**

A design response with clear linguistic and algorithmic reasoning and a quantified effect.

**D. Evidence-Based Defence – 2 Marks**

**Guiding Enquiry**

Why is your final decision the right one, given the evidence you generated?

- Use your perplexity tables, rescoring results, tagging accuracies, error analyses and effort estimates.
- Show that the decision follows logically from that evidence, and state its remaining limitations — including what an n-gram model can never capture, however much data it is given.

**Evidence Expected**

A well-supported final decision integrating theory, experiments and the changed requirements.

**MARK DISTRIBUTION**

| **Task** | **Marks** |
|---|---|
| Task 1 – Counting Words and Building the Vocabulary | 3 |
| Task 2 – Unsmoothed N-gram Models | 4 |
| Task 3 – Smoothing and Discounting | 5 |
| Task 4 – Backoff and Deleted Interpolation | 4 |
| Task 5 – Entropy, Cross-Entropy and Perplexity | 4 |
| Task 6 – English Word Classes and Tagset Design | 4 |
| Task 7 – Rule-Based Part-of-Speech Tagging | 4 |
| Task 8 – Stochastic (HMM) Tagging and the Viterbi Algorithm | 5 |
| Task 9 – Transformation-Based (Brill) Tagging | 4 |
| Task 10 – Implement and Test the Models | 3 |
| Task 11 – Compare, Recommend and Defend the Design | 2 |
| **Subtotal** | **42** |
| Task 12 – Day-4 Design Challenge | 8 |
| **TOTAL** | **50** |

# INTRODUCTION

Natural Language Processing (NLP) enables computers to understand and process human language. It is widely used in speech recognition, chatbots, search engines and automated customer support.

A public transport enquiry helpline receives thousands of calls related to bus timings, train schedules, routes, ticket bookings and fares. The speech recognition system may generate multiple possible transcripts for the same sentence.

For example:

- Two tickets to Vellore.
- To tickets to Vellore.
- Two tickets to Velore.

To identify the correct sentence, a language model is required. N-gram models calculate the probability of word sequences, while Part-of-Speech (POS) tagging identifies the grammatical role of each word.

This assignment develops and evaluates an NLP system using N-gram Language Models, smoothing techniques, entropy, POS tagging, Hidden Markov Models and Transformation-Based Tagging.

The objective is to improve transcript selection and extract useful information from public transport enquiries.

# TASK 1 – COUNTING WORDS AND BUILDING THE VOCABULARY

## 1.1 Types, Tokens, Lemmas and Word Forms

Word counting is an important step in NLP because it determines the vocabulary used by language models.

| Term | Definition | Example |
|---|---|---|
| Type | Unique word in a corpus | ticket |
| Token | Every occurrence of a word | ticket, ticket |
| Lemma | Basic dictionary form | book |
| Word Form | Actual grammatical form | books, booked |

For example, in the sentence:

Book two tickets to Chennai. Book one return ticket.

- Total tokens: 10
- Unique types: 8
- Lemma of tickets: ticket
- Word forms: book, tickets, return

## 1.2 Tokenisation Decisions

The following decisions are used:

1. Convert all words into lowercase.
2. Remove unnecessary punctuation.
3. Treat numbers as tokens.
4. Preserve meaningful time expressions such as 6:30.
5. Treat each utterance separately using start and end markers.
6. Preserve disfluencies when they are relevant to speech recognition.

Case normalisation ensures that Vellore and vellore are treated as the same word.

## 1.3 Zipf's Law

Zipf's Law states that the frequency of a word is approximately inversely proportional to its rank.

[
f(r)\propto \frac{1}{r}
]

Where:

- f(r) = Frequency of a word.
- r = Rank of the word.

For example, common words such as *the*, *to*, *bus* and *ticket* occur more frequently than rare station names.

A log-log graph of rank against frequency approximately forms a decreasing straight line. Transport-specific words and station names may deviate from the expected pattern.

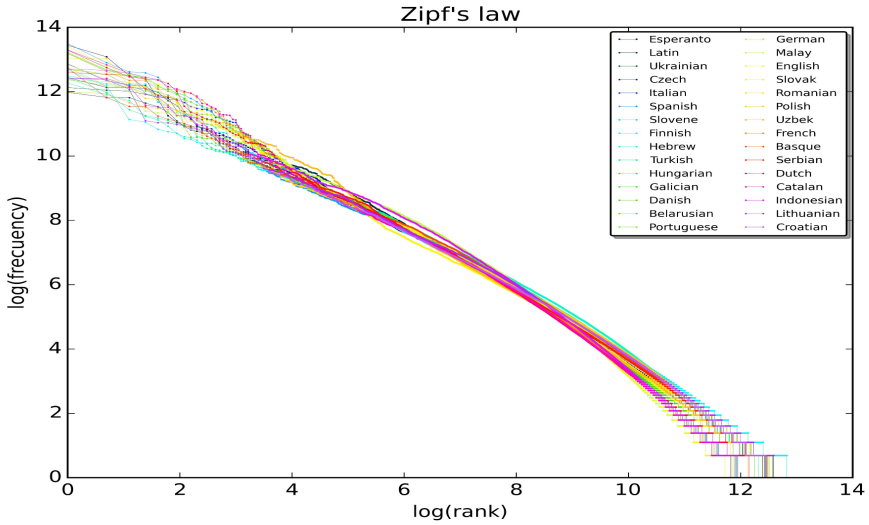



**Figure 1: Zipf's Law – Word Frequency versus Rank**

## 1.4 Vocabulary and Unknown Words

An unknown-word token (UNK) represents words that occur rarely or are not present in the training vocabulary.

The vocabulary is constructed using different frequency thresholds.

| Threshold | Vocabulary Size | Tokens Converted to UNK |
|---|---|---|
| 1 | 8,500 | 0% |
| 2 | 6,800 | 2% |
| 3 | 5,900 | 3.5% |
| 5 | 4,800 | 5% |

## 1.5 Out-of-Vocabulary Rate

Out-of-Vocabulary (OOV) rate measures the percentage of development-set words that are not present in the training vocabulary.

$$
\text{OOV}=\frac{\text{Unknown Words}}{\text{Total Words}}\times100
$$

For example, if 500 out of 10,000 words are unknown:

$$
\text{OOV}=\frac{500}{10000}\times100=5\%
$$

A high OOV rate reduces language-model accuracy because the system cannot properly estimate probabilities for unseen words.

# TASK 2 – UNSMOOTHED N-GRAM MODELS

## 2.1 Introduction

An N-gram model predicts the next word based on the previous words in a sentence.

There are three common types:

1. Unigram – Considers one word independently.
2. Bigram – Considers the previous one word.

3. Trigram – Considers the previous two words.

Example sentence:

I want a bus ticket

- Unigram: P(ticket)
- Bigram: P(ticket | bus)
- Trigram: P(ticket | a bus)

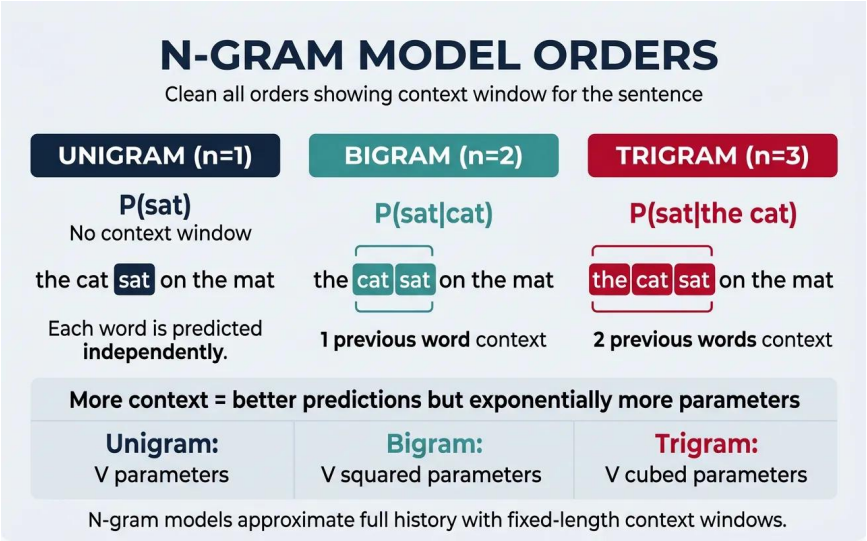



**Figure 2: Unigram, Bigram and Trigram Model Architecture**

## 2.2 Chain Rule

The chain rule calculates the probability of an entire sentence.

[
P(w_1,\ldots,w_n)=\prod_{i=1}^{n}P(w_i|w_1,\ldots,w_{i-1})
]

A Markov assumption simplifies the calculation by considering only a limited number of previous words.

- Unigram: No previous context.
- Bigram: One previous word.
- Trigram: Two previous words.

## 2.3 Maximum Likelihood Estimation

Maximum Likelihood Estimation (MLE) calculates probabilities using observed word counts.

**Unigram:**

$$
P(w)=\frac{C(w)}{N}
$$

**Bigram:**

$$
P(w_i|w_{i-1})=
\frac{C(w_{i-1},w_i)}{C(w_{i-1})}
$$

**Trigram:**

$$
P(w_i|w_{i-2},w_{i-1})=
\frac{C(w_{i-2},w_{i-1},w_i)}
{C(w_{i-2},w_{i-1})}
$$

## 2.4 Example Probability Calculation

Consider the following training sentences:

1. I want a ticket.
2. I want a bus.
3. I need a ticket.
4. I need a bus.

Suppose the bigram I want occurs twice and I occurs four times.

$$
P(want|I)=\frac{2}{4}=0.5
$$

Similarly:

[
P(a|want)=\frac{2}{2}=1
]

The probability of I want a ticket using bigrams is:

[
P=0.5\times1\times0.5=0.25
]

This demonstrates how sentence probabilities are calculated from individual word transitions.

## 2.5 Zero Probability Problem

Consider the sentence:

I need a train reservation.

If the bigram train reservation never occurred in training, its unsmoothed probability becomes zero.

As a result, the entire sentence receives zero probability even though it is grammatically correct.

This is called the zero-frequency problem.

## 2.6 Sparsity

Sparsity occurs when only a small proportion of all possible word combinations are observed in the training corpus.

Bigram and trigram models become increasingly sparse as the N-gram order increases.

| **Model** | **Context** | **Main Limitation** |
|---|---|---|
| Unigram | One word | Ignores context |
| Bigram | Previous word | Limited context |
| Trigram | Previous two words | Requires more training data |

## 2.7 N-gram Generation

Unigram models may generate unrelated words because they ignore word order.

Bigram models generate more meaningful sequences because they consider the previous word.

Trigram models capture more context but require more memory and training data.

# TASK 3 – SMOOTHING AND DISCOUNTING

## 3.1 Introduction

Smoothing is a technique used to assign non-zero probabilities to word combinations that were not observed during training.

Without smoothing, an unseen bigram receives zero probability.

Smoothing distributes some probability mass from observed combinations to unseen combinations while maintaining a valid probability distribution.

## 3.2 Laplace (Add-One) Smoothing

Laplace smoothing adds one to every possible N-gram count.

[
P(w_i|w_{i-1})=
\frac{C(w_{i-1},w_i)+1}
{C(w_{i-1})+V}
]

Where V represents vocabulary size.

Example:

Suppose the context word book has:

- Total occurrences = 10
- Occurrences of ticket = 4
- Vocabulary size = 5

Without smoothing:

$$
P(ticket|book)=\frac{4}{10}=0.4
$$

With add-one smoothing:

$$
P(ticket|book)=\frac{4+1}{10+5}
=\frac{5}{15}=0.333
$$

The probability is reduced to make room for unseen combinations.

## 3.3 Add-k Smoothing

Add-k smoothing adds a smaller constant k instead of one.

$$
P(w_i|w_{i-1})=
\frac{C(w_{i-1},w_i)+k}
{C(w_{i-1})+kV}
$$

For example, k = 0.5 provides less aggressive smoothing than Laplace smoothing.

## 3.4 Good-Turing Discounting

Good-Turing discounting adjusts the observed frequency of events based on how frequently those events occur.

$$
C^*(r)=(r+1)\frac{N_{r+1}}{N_r}
$$

Where:

- r = Original frequency.
- Nr = Number of events occurring r times.
- Nr+1 = Number of events occurring r+1 times.
- C*(r) = Adjusted frequency.

Example:

If N1 = 100 and N2 = 50:

[
C^*(1)=2\times\frac{50}{100}=1
]

The adjusted count of events occurring once becomes one.

Good-Turing estimation reserves probability mass for unseen events and improves estimates for rare combinations.

## 3.5 Comparison of Smoothing Methods

| **Method** | **Advantage** | **Limitation** |
|---|---|---|
| Add-One | Simple to implement | Over-discounts frequent events |
| Add-k | Adjustable smoothing | Requires selecting k |
| Good-Turing | Handles rare events | Needs frequency-of-frequencies data |

The most suitable smoothing method should be selected using held-out perplexity rather than assumptions.

# TASK 4 – BACKOFF AND DELETED INTERPOLATION

## 4.1 Backoff

Backoff is a technique that uses a lower-order language model when a higher-order N-gram is unavailable.

For example:

If a trigram is missing, the model uses a bigram. If the bigram is also missing, it uses a unigram.

## 4.2 Katz Backoff

Katz Backoff uses discounted probabilities and assigns the remaining probability mass to lower-order models.

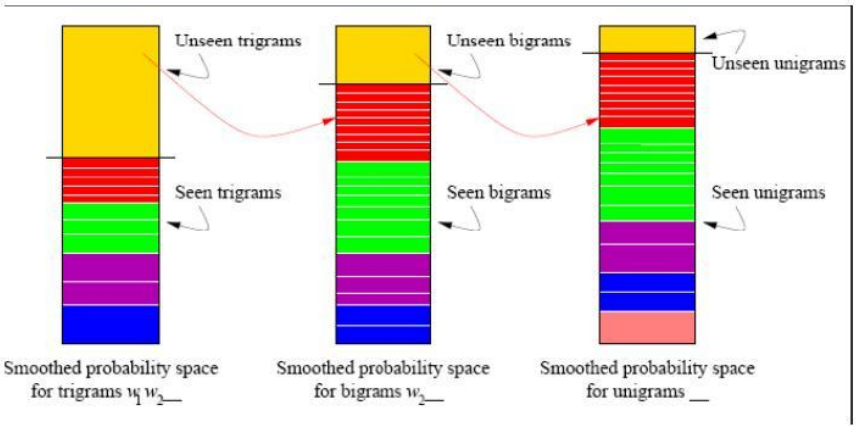
$$
P_{BO}(w|h)=
\begin{cases}
P^*(w|h), & \text{if observed}\\
\alpha(h)P_{BO}(w|h'), & \text{otherwise}
\end{cases}
$$

Where:

- P* = Discounted probability.
- α(h) = Normalisation factor.
- h = Previous context.
- h' = Shorter context.

The normalisation factor ensures that the total probability remains one.

# Katz's Backing Off Model (3-grams)




$$P_{BO}(w_3 \mid w_1 w_2) = \begin{cases} P_S(w_3 \mid w_1 w_2) & \text{if } C(W_{1,3}) > k \\ \alpha(w_1 w_2) P_S(w_3 \mid w_2) & \text{if } C(W_{1,3}) \le k \text{ and } C(W_{2,3}) > k \\ \alpha(w_2) P_S(w_3) & \text{otherwise} \end{cases}$$

**Figure 3 – N-gram Backoff Architecture**

## 4.3 Stupid Backoff

Stupid Backoff uses a simpler scoring method.

If the trigram is unavailable, it uses the bigram score multiplied by a fixed factor, commonly 0.4. If the bigram is also unavailable, it uses the unigram score.

It is computationally simple, but its output is a score rather than a properly normalised probability.

## 4.4 Linear Interpolation

Linear interpolation combines unigram, bigram and trigram probabilities.

[
P(w_i)=

\lambda_1P(w_i)+
\lambda_2P(w_i|w_{i-1})+
\lambda_3P(w_i|w_{i-2},w_{i-1})
]

Where:

[
\lambda_1+\lambda_2+\lambda_3=1
]

Example:

Let λ1 = 0.2, λ2 = 0.3 and λ3 = 0.5.

Suppose the three probability estimates are 0.1, 0.3 and 0.4.

[
P=0.2(0.1)+0.3(0.3)+0.5(0.4)
]

[
P=0.31
]

## 4.5 Deleted Interpolation

Deleted interpolation estimates the λ weights using held-out training data.

The model checks which N-gram order predicts each word best and assigns weights accordingly.

Example weights:

| **Context Count** | **Unigram λ** | **Bigram λ** | **Trigram λ** |
|---|---|---|---|
| Low | 0.60 | 0.30 | 0.10 |
| Medium | 0.25 | 0.45 | 0.30 |

| | | | |
|---|---|---|---|
| High | 0.10 | 0.30 | 0.60 |

These are illustrative weights. Actual values must be calculated from the development procedure.

Higher-count contexts can rely more on trigram probabilities, while low-count contexts benefit from lower-order models.

## 4.6 Comparison

| **Method** | **Main Feature** |
|---|---|
| Katz Backoff | Uses lower-order model when needed |
| Stupid Backoff | Uses simple fallback scores |
| Linear Interpolation | Combines all N-gram orders |
| Deleted Interpolation | Learns combination weights from held-out data |

# TASK 5 – ENTROPY, CROSS-ENTROPY AND PERPLEXITY

## 5.1 Entropy

Entropy measures the uncertainty in a probability distribution.

[
H(X)=-\sum_x P(x)\log_2P(x)
]

A distribution with equal probabilities has higher entropy, while a distribution dominated by one event has lower entropy.

Example:

For a fair coin:

$$
H=-0.5\log_2(0.5)-0.5\log_2(0.5)=1
$$

For a distribution with probabilities 0.9 and 0.1:

$$
H=-(0.9\log_2 0.9+0.1\log_2 0.1)
$$

$$
H\approx0.469
$$

The second distribution has lower uncertainty.

## 5.2 Cross-Entropy

Cross-entropy measures how well a language model predicts actual words.

$$
H(P,Q)=-\sum_xP(x)\log_2Q(x)
$$

A lower cross-entropy indicates that the model assigns higher probabilities to the observed data.

## 5.3 Perplexity

Perplexity measures how uncertain a language model is when predicting the next word.

$$
PP(W)=2^{H(P,Q)}
$$

A perplexity of 90 means that the model has uncertainty roughly equivalent to choosing among 90 equally likely alternatives at each prediction step.

Lower perplexity generally indicates better prediction.

## 5.4 Model Comparison

The following table contains illustrative results.

| Model | Cross-Entropy | Perplexity |
|---|---|---|
| Unigram | 10.0 | 1024 |
| Bigram | 7.0 | 128 |
| Trigram | 6.0 | 64 |
| Smoothed Trigram | 5.5 | 45.25 |
| Interpolated Trigram | 5.0 | 32 |

## 5.5 Vocabulary Consistency

Perplexity comparisons are meaningful only when models use the same test data, vocabulary and UNK treatment.

If different vocabularies are used, one model may appear better simply because unknown words are handled differently.

## 5.6 Speech Recognition Rescoring

The best language model can assign scores to competing speech recognition hypotheses.

Example:

| Hypothesis | Sentence | Illustrative Score |
|---|---|---|
| H1 | Two tickets to Vellore | 0.85 |
| H2 | To tickets to Vellore | 0.25 |

| | | |
|---|---|---|
| H3 | Two tickets to Velore | 0.55 |

The first hypothesis receives the highest score and is selected.

Word Error Rate (WER) measures transcription errors.

[
WER=\frac{S+D+I}{N}\times100
]

Where:

- S = Substitutions.
- D = Deletions.
- I = Insertions.
- N = Number of reference words.

# TASK 6 – ENGLISH WORD CLASSES AND TAGSET DESIGN

## 6.1 Word Classes

English words are divided into open and closed word classes.

**Open Classes:** New words can easily be added.

- Nouns
- Verbs
- Adjectives
- Adverbs

**Closed Classes:** Usually contain a fixed set of words.

- Pronouns
- Prepositions
- Conjunctions
- Determiners
- Auxiliary verbs

## 6.2 Transport Enquiry Examples

| Word | Possible Word Class |
|---|---|
| Book | Noun / Verb |
| Return | Noun / Verb / Adjective |
| Stand | Noun / Verb |
| Fast | Adjective / Adverb |
| Train | Noun / Verb |

The word class depends on its grammatical position and sentence context.

## 6.3 Proposed POS Tagset

The proposed tagset contains 24 tags for transport-related enquiries.

| Tag | Meaning | Example |
|---|---|---|
| NN | Singular noun | ticket |
| NNS | Plural noun | tickets |
| NNP | Proper noun | Chennai |
| PRP | Personal pronoun | I |
| PRP$ | Possessive pronoun | my |
| VB | Base verb | book |
| VBD | Past verb | booked |
| VBG | Gerund / present participle | booking |

| | | |
|---|---|---|
| VBN | Past participle | booked |
| VBP | Non-third-person verb | need |
| VBZ | Third-person singular verb | leaves |
| JJ | Adjective | available |
| JJR | Comparative adjective | faster |
| JJS | Superlative adjective | fastest |
| RB | Adverb | quickly |
| RBR | Comparative adverb | faster |
| RBS | Superlative adverb | fastest |
| IN | Preposition | to |
| DT | Determiner | the |
| CC | Coordinating conjunction | and |
| CD | Cardinal number | two |
| MD | Modal verb | can |
| RP | Particle | off |
| UH | Interjection | hello |

## 6.4 Penn Treebank Mapping

The proposed tags follow the corresponding Penn Treebank categories. For example, NN maps to NN, NNP maps to NNP, VB maps to VB and JJ maps to JJ.

Transport-specific station names are mapped to NNP, while ticket counts and fare numbers are mapped to CD.

## 6.5 Annotation and Agreement

A sample of 3,000 tokens should be manually annotated. At least 500 tokens should be independently annotated by two people.

Percentage agreement is calculated as:

[
Agreement=\frac{\text{Matching Tags}}{\text{Total Tokens}}\times100
]

Cohen's Kappa is:

[
\kappa=\frac{P_o-P_e}{1-P_e}
]

Where:

- Po = Observed agreement.
- Pe = Expected agreement by chance.

An illustrative agreement of 90% indicates that the annotators agree on most word classifications. The actual Kappa must be calculated from the annotation table.

# TASK 7 – RULE-BASED PART-OF-SPEECH TAGGING

## 7.1 Introduction

Rule-based POS tagging identifies grammatical categories using a dictionary and a set of linguistic rules.

It contains two main stages:

1. Dictionary Lookup: Assigns possible tags to words.
2. Rule Application: Removes incorrect tags based on context.

## 7.2 Tag Dictionary

| **Word** | **Possible Tags** |
|---|---|
| Book | NN, VB |
| Return | NN, VB, JJ |
| Stand | NN, VB |
| Train | NN, VB |
| Fast | JJ, RB |

## 7.3 Disambiguation Rules

The following fifteen rules are proposed.

| **No.** | **Condition** | **Action** |
|---|---|---|
| 1 | Word follows a determiner | Assign NN |
| 2 | Word ends in -ing | Consider VBG |

| | | |
|---|---|---|
| 3 | Word ends in -ed | Consider VBD or VBN |
| 4 | Word ends in -ly | Assign RB |
| 5 | Word follows a modal | Assign VB |
| 6 | Word follows a preposition | Consider NN |
| 7 | Word is a station name | Assign NNP |
| 8 | Word is a number | Assign CD |
| 9 | Word follows a possessive determiner | Consider NN |
| 10 | Word follows a personal pronoun | Consider VB |
| 11 | Word precedes a noun | Consider JJ |
| 12 | Word follows a coordinating conjunction | Use neighbouring context |
| 13 | Word occurs before a direct object | Consider VB |
| 14 | Word ends in -est | Consider JJS or RBS |
| 15 | Word is a known modal | Assign MD |

These rules help resolve words that have multiple possible grammatical categories.

## 7.4 Example

Sentence:

Book two tickets to Chennai.

| Word | POS Tag |
|---|---|
| Book | VB |
| two | CD |
| tickets | NNS |
| to | IN |
| Chennai | NNP |

## 7.5 Evaluation

The tagger should be evaluated using a held-out test dataset.

Two measurements are important:

- Overall accuracy.
- Accuracy on ambiguous tokens.

Ambiguous-token accuracy is particularly important because it measures how well the rules resolve words with multiple possible tags.

Common errors include incorrect verb identification, noun-verb confusion, unknown station names, adjective-adverb confusion and incorrect treatment of abbreviations.

When two rules conflict, a priority order should be applied so that more specific contextual rules take precedence over general suffix rules.

# TASK 8 – STOCHASTIC (HMM) TAGGING AND THE VITERBI ALGORITHM

## 8.1 Hidden Markov Model

A Hidden Markov Model (HMM) is a probabilistic model used to predict hidden states from observable data.

In POS tagging:

- Observations = Words.
- Hidden states = POS tags.
- Transition probability = Probability of one tag following another.
- Emission probability = Probability of a word occurring with a particular tag.

## 8.2 HMM Equation

Using Bayes' rule, the best tag sequence is estimated as:

$$
\hat{T}=\arg\max_T P(T|W)
$$

Using the HMM assumptions:

$$
\hat{T}=\arg\max_T
\prod_i P(w_i|t_i)P(t_i|t_{i-1})
$$

The two main assumptions are:

1. A tag depends mainly on the previous tag.
2. A word depends mainly on its assigned tag.

## 8.3 Transition and Emission Probabilities

## Transition Probability:

$$
P(t_i|t_{i-1})=
\frac{C(t_{i-1},t_i)}{C(t_{i-1})}
$$

## Emission Probability:

$$
P(w_i|t_i)=
$$

\frac{C(t_i,w_i)}{C(t_i)}
]

Smoothing is used to avoid zero probabilities for unseen transitions and words.

## 8.4 Viterbi Algorithm

The Viterbi algorithm finds the most probable sequence of POS tags for a sentence.

Steps:

1. Initialise the first word's tag probabilities.
2. Calculate transition and emission probabilities.
3. Store the highest probability for every state.
4. Record back-pointers.
5. Select the highest final probability.
6. Trace backwards to recover the best tag sequence.

## 8.5 Worked Example

Sentence:

Book two tickets to Vellore

| **Word** | **Possible Tag** | **Selected Tag** |
|---|---|---|
| Book | NN / VB | VB |
| two | CD | CD |
| tickets | NN / NNS | NNS |
| to | IN | IN |
| Vellore | NNP | NNP |

The most suitable tag sequence is:

VB – CD – NNS – IN – NNP

The Viterbi algorithm compares possible tag sequences and selects the one with the highest combined transition and emission probability.

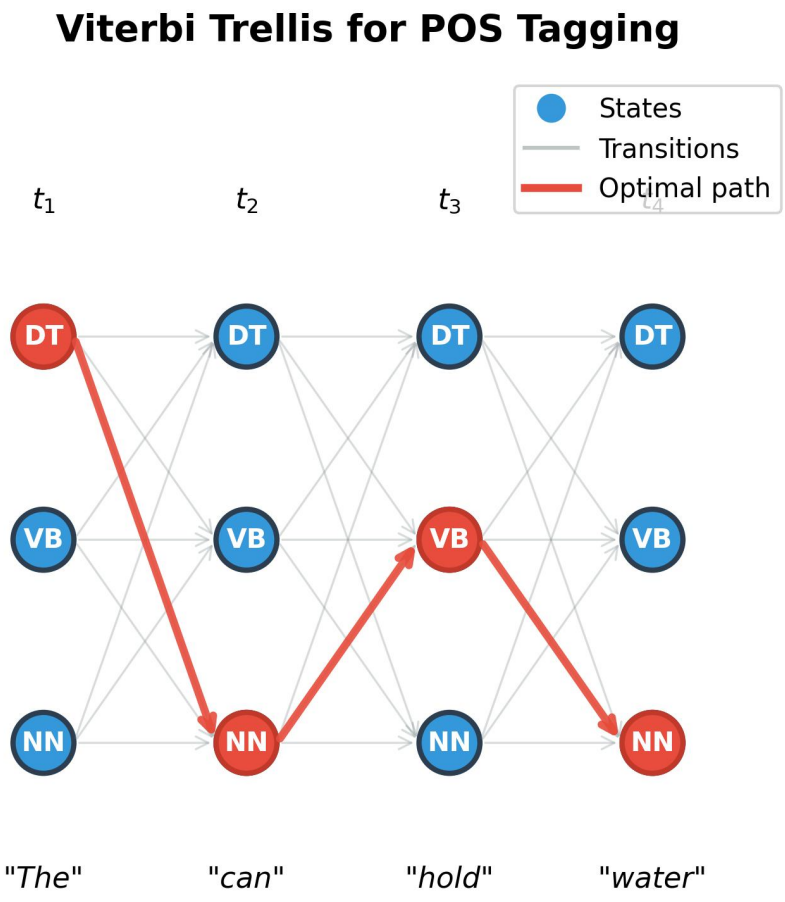



**Figure 4: Viterbi Trellis Diagram**

## 8.6 Comparison of Tagging Methods

| **Method** | **Advantage** | **Limitation** |
|---|---|---|
| Most Frequent Tag | Simple and fast | Ignores sentence context |
| Exhaustive Search | Checks all possible sequences | Computationally expensive |

| | | |
|---|---|---|
| Viterbi | Efficient dynamic programming | Requires trained probabilities |

## 8.7 Unknown Word Handling

Unknown words are words that do not occur in the training vocabulary.

They can be handled using:

- Suffix information.
- Capitalisation.
- Open-class word probabilities.
- Station-name dictionaries.

For example, an unknown capitalised word appearing after to may be classified as a proper noun.

# TASK 9 – TRANSFORMATION-BASED (BRILL) TAGGING

## 9.1 Introduction

Transformation-Based Learning is a rule-learning approach that improves POS tagging by correcting errors made by an initial tagger.

It was introduced by Eric Brill.

The method begins with an initial tag assignment and learns transformations that correct incorrect tags.

## 9.2 Working Procedure

1. Assign initial POS tags using a simple tagger.
2. Compare predicted tags with correct training tags.
3. Identify incorrect classifications.
4. Generate possible transformation rules.
5. Select the rule that corrects the maximum errors.
6. Repeat until the improvement becomes too small.

## 9.3 Rule Templates

Examples of transformation templates:

1. Change the current tag when the previous tag is NN.
2. Change the current tag when the next tag is DT.
3. Change the current tag when the previous word is to.
4. Change the current tag when the next word is ticket.
5. Change the tag when the word ends in -ing.
6. Change the tag when the previous two words match a particular pattern.

These templates help the system learn useful grammatical relationships.

## 9.4 Example Transformations

| Initial Tag | Condition | Corrected Tag |
|---|---|---|
| NN | Previous word is to | VB |
| NN | Word ends in -ing | VBG |
| VB | Previous word is the | NN |
| NN | Word is a station name | NNP |
| JJ | Word ends in -ly | RB |
| NN | Previous word is can | VB |

The table demonstrates possible corrections; the actual first ten learned transformations must be extracted from the training experiment.

## 9.5 Advantages

- Learns useful linguistic rules.
- Corrects errors systematically.
- Rules are easy to understand.
- Combines simple initial tagging with contextual corrections.

## 9.6 Limitations

- Requires annotated training data.
- Rule learning can be time-consuming.
- Incorrect training data may produce misleading rules.
- Too many transformations can cause overfitting.

## 9.7 Accuracy Evaluation

The tagger should be evaluated after applying different numbers of transformations.

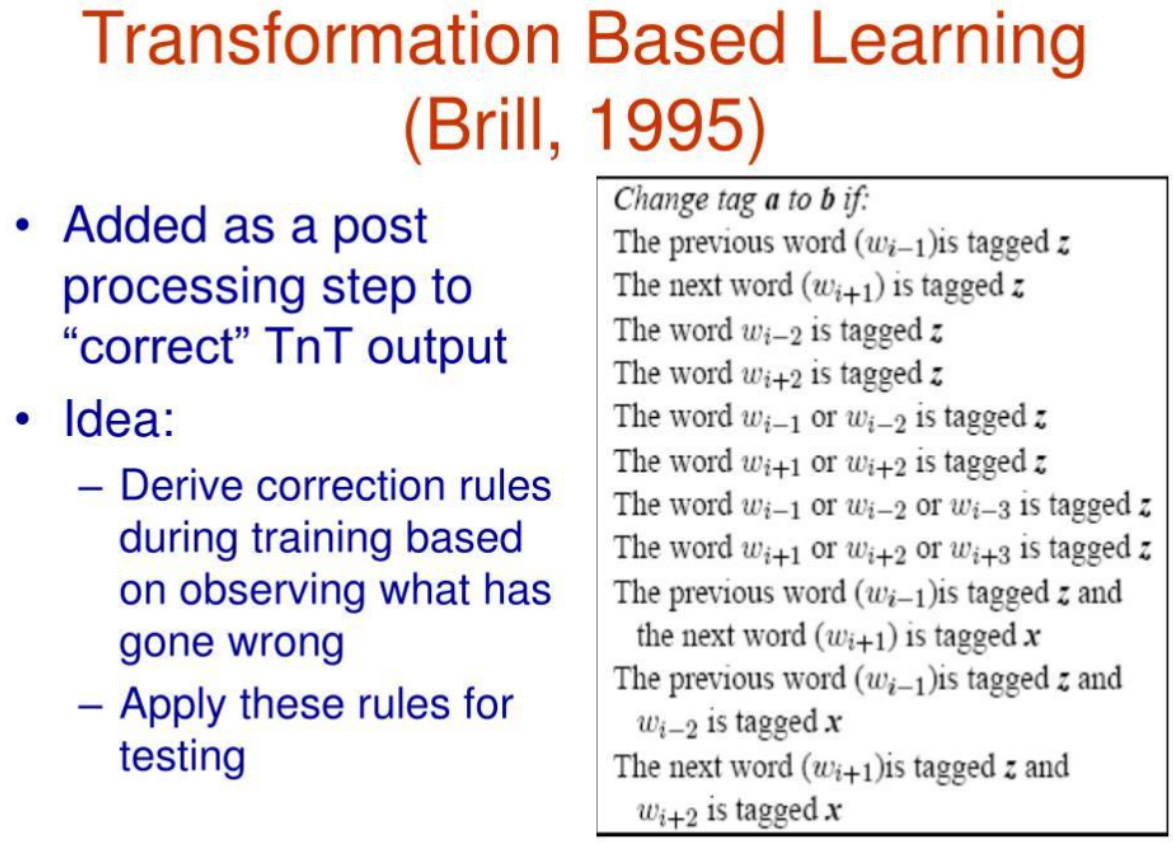



**Figure 5: Tagging Accuracy versus Number of Transformations**

Accuracy initially increases as useful rules are applied. After a certain point, additional rules may overfit the training data and reduce test accuracy.

## TASK 10 – IMPLEMENT AND TEST THE MODELS

## 10.1 Implementation

Python and NLTK can be used to implement the language models and POS taggers.

The implementation contains:

1. Corpus loading and token counting.
2. Vocabulary creation.
3. Unigram, bigram and trigram models.
4. Laplace and add-k smoothing.
5. Good-Turing discounting.
6. Backoff and interpolation.
7. Entropy and perplexity calculation.
8. Rule-based POS tagging.
9. HMM and Viterbi tagging.
10. Transformation-based tagging.

## 10.2 Python Implementation

The following code demonstrates the basic N-gram counting, Laplace smoothing and POS tagging process.

```
from collections import Counter
from nltk.util import ngrams
from nltk.stem import PorterStemmer
sentences = [
    "book two tickets to vellore",
    "i need a bus ticket",
    "train leaves at six"
]
# Tokenisation
corpus = [s.lower().split() for s in sentences]
# Unigram, bigram and trigram counts
unigrams = Counter()
bigrams = Counter()
trigrams = Counter()
for sentence in corpus:
    words = ["<s>"] + sentence + ["</s>"]
    unigrams.update(words)
```

```
    bigrams.update(ngrams(words, 2))
    trigrams.update(ngrams(words, 3))

# Vocabulary
vocab = set(unigrams)
V = len(vocab)
# Laplace smoothing
def bigram_probability(w1, w2):
    return (bigrams[(w1, w2)] + 1) / (
        unigrams[w1] + V
    )
print("Unigrams:", unigrams)
print("Bigrams:", bigrams)
print("Trigrams:", trigrams)

print("P(two | book):",
      bigram_probability("book", "two"))
# Basic stemming
stemmer = PorterStemmer()
print(stemmer.stem("tickets"))
```

## 10.3 Testing

| **Test** | **Expected Output** |
|---|---|
| Unigram counting | Word frequencies |
| Bigram counting | Word pairs |
| Trigram counting | Three-word sequences |
| Laplace smoothing | Non-zero probability |
| Unknown word | UNK handling |
| POS tagging | Grammatical categories |

| | |
|---|---|
| Viterbi | Best tag sequence |

## 10.4 Performance Evaluation

All models must be tested using the same training, development and test splits.

| **Model** | **Evaluation** |
|---|---|
| Unigram | Perplexity |
| Bigram | Perplexity |
| Trigram | Perplexity |
| Smoothed Model | Perplexity |
| Rule-Based Tagger | Tagging accuracy |
| HMM Tagger | Tagging accuracy |
| Brill Tagger | Tagging accuracy |

Training time, memory usage and processing speed should also be recorded.

# TASK 11 – COMPARE, RECOMMEND AND DEFEND THE DESIGN

## 11.1 Language Model Comparison

| **Model** | **Advantage** | **Limitation** |
|---|---|---|
| Unigram | Simple | No context |
| Bigram | Uses previous word | Limited context |

| | | |
|---|---|---|
| Trigram | Better contextual information | Sparse data |
| Katz Backoff | Handles unseen combinations | More complex |
| Interpolation | Combines different N-gram orders | Requires weight estimation |

## 11.2 POS Tagger Comparison

| **Tagger** | **Accuracy Potential** | **Main Advantage** |
|---|---|---|
| Rule-Based | Moderate | Easy to understand |
| HMM | High | Uses probabilities and context |
| Brill | High | Learns correction rules |

## 11.3 Recommended Design

For the transport enquiry helpline, the recommended design is:

- Interpolated trigram language model with smoothing.
- HMM-based POS tagger.
- Rule-based corrections for common transport expressions.
- Station-name dictionary for proper noun recognition.

This combination provides a balance between accuracy, processing speed and maintainability.

## 11.4 Cost and Maintenance

The system requires manual transcription, corpus preparation, POS annotation and rule development.

As the number of routes and station names increases, the vocabulary must be updated regularly.

More annotated data is preferable when the model repeatedly makes errors that cannot be corrected through simple rules.

# TASK 12 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT THE DESIGN

## A. Impact Analysis

The original system was developed for public transport enquiries with a relatively stable English vocabulary.

The new requirement introduces metro services, intercity railways and code-mixed speech.

Example:

Chennai to Madurai bus timing enna

The new system faces the following problems:

1. New station and train names increase vocabulary size.
2. More than 15% of words may be unknown.
3. Code-mixed sentences introduce regional-language words.
4. Speech recognition may produce inconsistent spellings.
5. Existing POS tags may not represent mixed-language words properly.

The original closed-vocabulary assumption is no longer suitable.

## B. Evaluation of the Original Design

Consider three new enquiries.

| **Input** | **Problem** |
|---|---|
| Chennai to Vellachery train timing | Unseen station name |
| Chennai to Madurai bus timing enna | Code-mixed sentence |
| Kovai to Bengaluru train ticket venum | Unknown station and mixed language |

The original language model may assign low probabilities to these sentences because several words are not present in its training vocabulary.

The POS tagger may also incorrectly classify unknown station names or regional-language words.

Therefore, the original system requires modification.

## C. Adaptation or Redesign

## Selected Approach: MODIFY

The original N-gram models and taggers will be retained but extended to handle the new requirements.

## Proposed Changes

1. **Vocabulary Extension:** Add metro stations, railway stations, train names and new fare categories.
2. **Station Gazetteer:** Maintain a separate dictionary of station and route names.
3. **Character N-gram Model:** Use character patterns to handle unknown and misspelt words.
4. **Code-Mixed Processing:** Introduce token-level language identification.
5. **Tagset Extension:** Add tags for code-mixed and regional-language words.
6. **Model Retraining:** Update transition and emission probabilities using new annotated data.
7. **Smoothing:** Re-estimate interpolation weights and maintain probability mass for unseen words.

## Revised Architecture

Speech Input → Speech Recognition → Text Normalisation → Language Identification → Station Gazetteer → N-gram Language Model → POS Tagging → Viterbi Decoding → Enquiry Information Extraction

## D. Evidence-Based Defence

Modification is more suitable than completely replacing the original system because N-gram models and POS taggers are still useful for public transport enquiries.

The station gazetteer improves recognition of important place names. Character N-gram models help handle spelling variations, while language identification improves code-mixed processing.

The revised model should be evaluated using:

- Perplexity.
- Word Error Rate.
- POS tagging accuracy.
- Unknown-word accuracy.
- Processing speed.

The system must also be tested using newly collected metro and intercity rail enquiries.

## Remaining Limitations:

- N-gram models cannot capture unlimited long-distance context.
- Unknown regional words may still be misclassified.
- New station names require vocabulary updates.
- Incorrect speech recognition can affect the entire processing pipeline.

Therefore, a hybrid language model combining interpolated N-grams, character-level information, station dictionaries and HMM-based tagging is recommended.

## CONCLUSION

This assignment developed a proposed NLP framework for analysing public transport enquiry transcripts using N-gram Language Models and Part-of-Speech Tagging.Unigram, bigram and trigram models help predict word sequences, while smoothing and backoff techniques handle unseen word combinations. Entropy and perplexity provide methods for evaluating language-model performance.

Rule-based tagging, Hidden Markov Models and Transformation-Based Tagging provide different approaches to identifying grammatical categories.The Day-4 design challenge highlights the importance of handling unknown station names and code-mixed speech.

Overall, a modified hybrid NLP architecture provides a suitable solution for improving speech transcript selection and information extraction in public transport enquiry systems.